2024 IEEE 4th International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA) | 979-8-3503-6226-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/VLSISATA61709.2024.10560174

# Design and Implementation of an Efficient 32-bit Fixed-Point Newton-Raphson Division-based Reciprocal Computing Unit

Chiranthgynan S Aragula, Budi Preethi, Chinthala Ramesh
*Department of Electronics and Communication Engineering,*
*Amrita School of Engineering, Bengaluru,*
*Amrita Vishwa Vidyapeetham, India*
Email: {bl.en.u4ece20043, 35}@bl.students.amrita.edu, {c_ramesh}@blr.amrita.edu

***Abstract*—Reciprocal operations play a critical role in digital signal processing (DSP), specifically in the execution of linear equation solving and Fourier analysis. For DSP architectures to support functions like adaptive filtering, channel equalization, and spectral estimation, efficient reciprocal computation is vital. Reciprocal computations frequently employ the Newton-Raphson algorithm, which is flexible concerning both floating-point and fixed-point formats. This paper proposes a digital design architecture of the reciprocal computing unit using the Newton Raphson method, which is modeled, and implemented using Verilog HDL, and the Xilinx Vivado Tool. Enhanced accuracy and substantial reductions in error rates are successfully attained using the proposed design. The average mean error rate of roughly 0.0264% is associated with converging values. The design uses optimized resources by decreasing the required logic elements by an average of 15.76%. The results of our study emphasize the capability of our design and methodology to improve both computational efficiency and accuracy in DSP applications.**

***Index Terms*—Newton-Raphson algorithm, Fixed-point, Channel equalization, Adaptive filtering, Spectral estimation**

## I. Introduction

Reciprocal computation, particularly the determination of inverse numbers, holds pivotal importance across numerous domains like multi-input multi-output (MIMO) communication systems, Digital Signal Processing (DSP), and linear algebra. Reciprocal calculation usually takes a variable and in a few cases more time due to the nature of the values such as the numerator being too big and the denominator being too small or vice-versa. Therefore precise and faster reciprocal computation plays a vital role in advancing technologies reliant on mathematical computations that use reciprocal calculations quite frequently.

The existing literature on the inverse computation of a real number represented using a fixed-point format is limited, leaving notable gaps in addressing critical requirements such as variable latency, minimal resource utilization, and high precision. Existing methodologies often rely on floating-point arithmetic, leading to higher computational complexity and memory requirements incompatible with resource-constrained environments. Additionally, inadequate attention is given to variable latency and minimal resource utilization, which are critical in modern computational paradigms. Addressing these challenges in reciprocal computation is crucial due to its pivotal role in real-world applications and the demand for efficient methods.

Our goal is to create a digital design system for computing reciprocal using the Newton-Raphson method within the constraints of fixed-point arithmetic, focusing on high precision while minimizing area utilization. The proposed digital design architecture uses serial architecture instead of parallel architecture, as parallel architecture has the drawback of using more resources does not suit the iterative computational methods to perform computation on a single value, and further leads to more power consumption. Therefore, we propose our approach for enhancing reciprocal computation through the Newton-Raphson method in fixed-point arithmetic, aiming to optimize both precision and area utilization using a 32-bit fixed-point representation and a serial architecture respectively.

We have structured the rest of this brief as follows: As part of the literature review, Section II explores enhancements made in various parameters such as area, latency, precision, converging speed, and resources, alongside the methodologies utilized to achieve these advancements. In Section III, we outline the flowchart demonstrating the functionality of the Newton Raphson Divison (NRD) to find the reciprocal of a given real number in fixed point format. In Section IV, we present an overview of the NRD Algorithm's logic and the implementation details for the digital design architecture. In Section V, we present experimental data including area, speed, throughput, accuracy of convergence, review of errors, and power consumption. Finally, in Section VI, we summarize the main conclusion of this brief.

## II. Literature Survey

In the realm of hardware implementation for numerical computations, advancements have been rapidly evolving, particularly within FPGA (Field-Programmable Gate Array) platforms. These developments target essential aspects such as area optimization, latency reduction, and precision enhancement to cater to diverse application requirements across various domains. In recent years, several innovative methodologies have emerged, each addressing specific challenges in computational efficiency such as accuracy, precision, less number of iterations.

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.

The 128-bit modular inverse unit using FPGA technology, showcased in [1], employs the Extended Euclidean Algorithm to significantly enhance the performance metrics. The design presented achieved a minimal critical path delay of 16.84 ns, allowing for an operational frequency of up to 72.7 MHz. Reporting a massive 7157 logic element and maintaining a power dissipation of 231.41 mW. Alternatively, the strategy described in [2] successfully reduces the need for conventional remainder calculations, doing so without adding extra latency. This method shows performance improvement, while moderately increasing the area and critical path, and enhancing power efficiency.

While implementing it in hardware, the practical application of the Reciprocal Square Root (RSR) algorithm was considered feasible on a low-cost FPGA device [3]. The paper reported the usage of Look-Up-Tables (LUTs) was below 1000, coupled with a latency of only 3 clock cycles, aiming to enhance the computational efficiency as discussed in [2]. The approach integrates a distinctive variable latency Goldschmidt iteration technique with an innovative rounding method to minimize the need for complex remainder calculations.

A scaling-less Newton-Raphson method in fixed-point format is used to compute the inverse square root in computational architecture in [4]. This novel approach eliminates coefficient scaling and memory storage, increasing operating frequencies. Furthermore, a novel rounding method [5] utilizes parallel remainder estimation for both Goldschmidt and Newton-Raphson (NR) algorithms, combining improvements in latency and precision. Within the scope of hardware optimization, [6] the focus is on efficient root function, inverse, and inverse root algorithms for complex matrix operations in MIMO communication systems.

Hardware implementation methodologies for numerical computations on FPGA platforms are constantly evolving, highlighting efforts to optimize area, latency, and precision. Each approach, from compact and power-efficient designs to latency-optimized algorithms and precision-enhancing methods, has its own impact on computational efficiency across application domains. Further advances in hardware acceleration techniques without compromising precision, accuracy, area, and power consumption could unlock new frontiers in high-performance computing, enabling novel applications and driving industry innovation.

## III. Design Methodology

Two prominent gaps in the literature: the lack of a comprehensive convergence analysis for the Newton-Raphson division and the insufficient use of fixed-point representation for reciprocal computations led to the decision to make reciprocal computation the primary emphasis of this study. Although reciprocal computation in fixed-point arithmetic is fundamentally important in many computational domains, the exploration of effective ways to maintain a low error while limiting the power and resource utilization to a comparably lower value is still relatively unexplored. The study aims to close the critical gap by preserving effects concerning parameters that are important in terms of hardware implementation while maintaining a modest error, with the purpose of analyzing the efficiency of the Newton-Raphson method for reciprocal computation. In the subsequent sections of this methodology, we outline the specific research objectives, research design, data collection methods, and data analysis techniques employed in this quantitative study.

### A. Newton Raphson Division

In this study, we utilized the Newton-Raphson technique, recognizing its constraints regarding initial approximations and potential divergence for specific values. Despite its requirement for a close initial guess and the computational overhead of calculating the inverse derivative, the method's advantages justify its selection. The Newton-Raphson method offers fast convergence when the initial guess is closer to the actual value, making it efficient for finding accurate solutions, especially when the function is well-behaved near the reciprocal. Its iterative improvement and versatility ensure applicability across a wide range of mathematical problems, while its robustness and adaptability make it a reliable choice for practical scenarios. We were able to successfully use the benefits of the Newton-Raphson technique to get around the challenges presented by earlier studies on reciprocal computation by carefully weighing these variables.

The fixed-point data representation is preferred over the floating-point representation to reduce the convergence time and design area, and the 32-bit data width provides sufficient precision. C program is used to generate the test vectors (input vectors and corresponding reference outputs).

A non-pipelined design architecture is used instead of a pipelined one to reduce design complexity, such as synchronization issues that naturally arise in designs performing iterative computations. The non-pipelined design also requires fewer resources compared to the pipelined design.

In our research, we utilized Xilinx Vivado as the primary tool for the RTL development of the digital design architecture of the reciprocal finder, and test bench to read the test vectors drive the design, and simulate to obtain the input and output waveforms. These test waveforms are used to test its functionality. The computed values are stored in a file and analyzed further to find the latency, accuracy, errors, etc. The Xilinx Vivado tool is used to synthesize and obtain the resources utilized by the design. With Vivado's intuitive design environment and powerful synthesis tools, designers can efficiently create and optimize FPGA designs, streamlining the development process and achieving high-performance results.

### B. Mathematical Calculations

The Newton-Raphson method stands as a robust iterative tool for pinpointing the roots of functions. In the specific context of computing reciprocals, characterized by

$$f(x) = \frac{1}{x} - D \tag{1}$$

where D represents the input operand, The value of $D$ is scaled to select the initial guess $X_0$ to be in the range $0.5 \leq$

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.

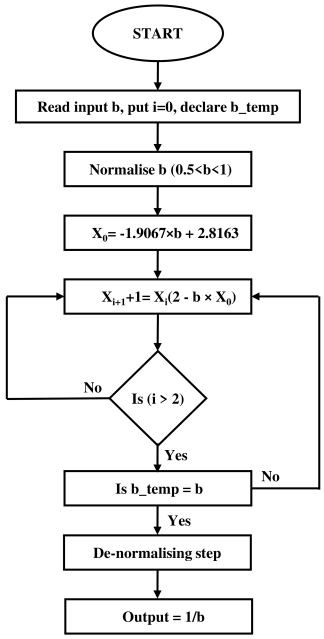



Fig. 1: Flow Chart for Computing 1 by b Fraction Using Newton-Raphson Method

$X_0 \leq 1$ so that a minimal number of iterations are needed for computation. The iteration unfolds as:

$$X_{i+1} = X_i - \frac{f(X_i)}{f_1'(X_i)} \tag{2}$$

Here, $f_1'(X_i)$ symbolizes the first derivative of $f(X_i)$

$$X_0 = \frac{48}{17} - \frac{32}{17}D \tag{3}$$

The Newton-Raphson iteration gives

$$X_{i+1} = X_i(2 - D \times X_i)$$

Achieving swift convergence hinges on the sharp selection of the initial guess $X_0$. Given the constraint that D must fall within it's paramount to align $X_0$ within this interval. If necessary, adjustments to D can be made to ensure compliance with this range.

In this study, we utilize the Newton-Raphson iterative division method to determine the reciprocal of a specified numerical value. Figure 1 illustrates the flow chart representing this approach. This methodological strategy begins with the systematic initialization of crucial parameters and the acquisition of the input variable, denoted as 'b'. Subsequently, we set the iteration counter '$i$' to zero and declare a temporary variable '$b_{\text{temp}}$', essential for subsequent computations. An integral step in this process involves the normalization of the divisor '$b$' to ensure it falls within the specified range of 0.5 to 1.0, a prerequisite crucial for the algorithm's convergence. We initiate the process by establishing our initial approximation, '$X_0$', calculated through a meticulously crafted formula designed to optimize convergence within the Newton-Raphson framework. Subsequent iterations, represented by '$X_{i+1}$', are then computed based on the preceding approximation '$X_i$' and the divisor '$b$'. This iterative refinement continues until convergence is attained, guided by a predefined criterion typically involving a comparison with a threshold value or a predetermined maximum number of iterations. Throughout the iterative process, we exercise vigilance, conducting checks to ensure convergence is achieved within the initial two iterations. Should further iterations be required, the algorithm continues until convergence is reached. Moreover, we validate the consistency of the divisor '$b$' throughout the iterative procedure, ensuring it remains within the specified range. This methodological framework epitomizes a systematic and methodical approach to computing the reciprocal of a numerical value through the utilization of the Newton-Raphson iterative division method.

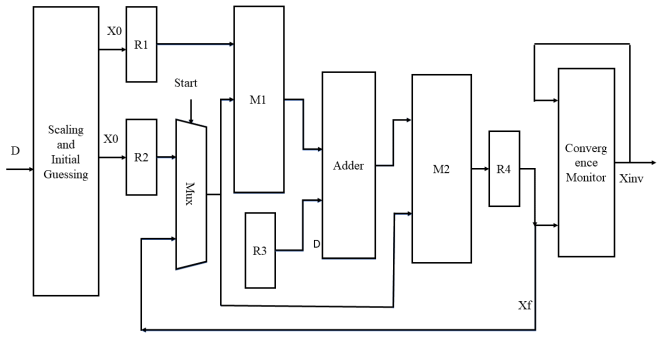



Fig. 2: Design Structure of 1 by x Fraction Finder

## IV. Implementation

### A. Architectural Overview of the NRD Algorithm

The architectural depiction explains the Newton-Raphson-based reciprocal computation methodology, characterized by its serial architecture for optimal computational efficiency. Initiation of the computational process is marked by a pulsed control signal, signifying the commencement of iterative iterations. In the series of iterations, a preprocessing step as depicted in scaling Figure 2 is implemented to rectify instances if anywhere the divisor (b) exceeds the predefined range of 0.5 to 1, ensuring numerical stability and convergence.

The architectural flow intricately handled the storage and manipulation of data. One output is meticulously stored in a designated register R1, facilitating its direct integration into a dedicated M1 multiplier module. Simultaneously, the secondary output R2, in conjunction with feedback from the preceding iteration's $X_f$ value, is directed into a multiplexer for seamless processing.

Subsequently, the resultant value, denoted as $X_i$, undergoes a series of intricate transformations, including multiplication by $D$ to yield $D \times (X_i)$. A specialized subtractor/adder module, responsive to the sign of input, dynamically adjusts the computational flow, either subtracting or adding based on the input polarity. This refined value is seamlessly fed back into the multiplier, culminating in the precise computation of $X_i$.

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.

Throughout the iterative process, characterized by meticulous scrutiny, the interplay between $X_f$ values across successive iterations is meticulously analyzed. Ultimately, the culmination of iterations yields the coveted final output $X_{\text{inv}}$, serving as the definitive representation of the computed reciprocal value.

The Verilog code provided defines a convergence criterion based on the stability of a signal over multiple clock cycles. Triggered by the positive edge of a clock signal, it assesses the behavior of a signal. If the signal maintains its value for more than two consecutive iterations, indicating convergence, it flags this condition by setting a convergence indicator. Otherwise, it updates an iteration counter and a separate counter variable. When a new input is detected, both counters are reset to zero to begin the convergence evaluation over again.

## V. Results and Discussions

The proposed digital sub-system design to find the reciprocal of a number represented in the fixed-point format is modeled using Verilog and verified for its functionality using Vivado 2023.1. The simulation results are used to find the latency, throughput, and accuracy of the reciprocal finder. The synthesis of the proposed design is carried out using Xilinx Vivado 2023.1. Vivado gives implementation reports that provide detailed insights into critical parameters such as critical path analysis, resource utilization, and estimated power consumption. These metrics contribute to a comprehensive evaluation of the digital sub-system's efficiency and scalability, enabling us to gauge its real-world applicability. The discussion interprets these results in the context of the research objectives, shedding light on how the proposed design surpasses or aligns with existing approaches. Moreover, practical implications for digital systems are explored, providing a roadmap for potential optimizations and future applications. The combination of simulation and synthesis results forms the cornerstone of our evidence-backed analysis, ensuring a thorough understanding of the proposed efficient implementation of fixed-point Newton-Raphson division for reciprocal calculation in digital systems. We utilized the C code to generate random numbers between 0 and 500 using the rand() function in order to verify the accuracy of our approach. The information was entered into a notepad file. Using the fixed point format, the corresponding reciprocal for each of the 1024 inputs was found, and the accuracy was adjusted to 16 digits after decimal and the data set is generated. The functionality of the proposed digital design architecture of the reciprocal finder is verified using the above data set. The latency, convergence efficiency, and error analysis are as follows.

### A. Results

*1) Latency:* Latency is determined by measuring the number of clock cycles elapsed between sending an input to an IP/system and receiving the corresponding output. This process involves tracking the propagation of signals through the system's logic elements and registers. By counting the clock cycles, one can assess the responsiveness of the system to inputs. In Vivado behavioral simulation, the average latency recorded stands at 5 clock cycles. This implies that, on average, the system requires 5 clock cycles to react to an input and generate the corresponding output. The latency of a digital design performing iterative computation to produce the final output is given by Eq. 4;

$$L = m * n \tag{4}$$

where $m$ is the number of iterations and $n$ is the number of cycles in each iteration. The $m$ value depends on the magnitude of the input values, large or small, and whether the inverse can be represented using the given fractional bit length. The $n$ value depends on the type of architecture, such as pipelined or non-pipelined, for pipelined architectures is equal to the number of pipeline stages, for non-pipelined it is 1. Therefore, the latency of a non-pipelined architecture is equal to the $m$.

*2) Frequency:* Frequency is calculated based on the reciprocal of the time difference between arrival time $T_i$ and Worst Negative Slack ($WNS_i$) in a digital circuit as shown in Eq. 5.

$$f = \frac{1}{T_i - WNS_i} \tag{5}$$

Non-reciprocal devices such as isolators and circulators, are crucial in telecommunications and radar systems for signal routing and noise reduction. They also play essential roles in phased array antennas, frequency converters, RF switches, and biomedical applications like MRI and quantum information processing.

*3) Throughput:* Throughput is determined as the rate at which the digital sub-system processes input data and produces corresponding outputs. From Eq. 6 a throughput of 10.10 MOPS is achieved, using a clock frequency of 50.50 MHz and a processing latency of 5 clock cycles.

$$T = f/ANCD \tag{6}$$

where $T$ is the throughput, $f$ is the frequency and ANCD is the average number of cycles per completing one division operation. The proposed design is operating with a T of 10.10 MOPS. By carefully optimizing the design of the digital sub-system with techniques like pipeline insertion and clock gating, the system ensures efficient processing of input data for specific applications. Balancing clock frequency and processing latency maximizes throughput while maintaining performance and reliability. This optimized throughput is especially useful for tasks like signal processing, image recognition, and real-time data analysis, where fast and effective computation is crucial.

*4) Convergence Analysis:* The effectiveness of the Newton-Raphson method hinges on an initial guess close to the actual reciprocal, allowing for rapid, quadratic convergence. However, divergent or slow convergence outcomes arise when this proximity condition isn't met. Factors contributing to such behavior include the local behavior of the function near the reciprocal, the existence of multiple reciprocals leading to unpredictable outcomes, the sensitivity of the method to initial guesses, numerical errors, and issues inherent to the iterative

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Comparison of Performance Metrics

| **Approach** | **Architecture type** | **Data Type** | Precision | **Latency** | **Throughput** | **Frequency** |
|---|---|---|---|---|---|---|
| Umut Kucukkabak [7] | Pipelined | Floating | 52 | 11 cycles | - | 196 MHz |
| Peter Malik [8] | Non-Pipelined | Floating | 32 | - | 13.43 MOPS | 67.150 MHz |
| Erwan, Arzel [4] | Pipelined | Fixed-point | 16 | 31 cycles | - | 740 MHz |
| Our design | Non-Pipelined | Fixed-point | 32 | 5 cycles | 8.42 MOPS | 50.50 MHz |

TABLE II: Iteration Performance and Error Evaluation

| **Iteration** | **10** | **11** | **12** | **13** | **14** | **15** |
|---|---|---|---|---|---|---|
| Avg latency | 4.922 | 5.028 | 5.57 | 5.5029 | 5.504 | 5.577 |
| Convergency(%) | 5.577 | 13.76953125 | 14.35546875 | 16.69921875 | 21.6796875 | 21.2890625 |
| Min error | 0.000267 | 1.9E-06 | 0.00027 | 1.9E-06 | 0.00026 | 1.9E-06 |
| Max error | 0.000580 | 0.00058 | 0.00028 | 0.00030 | 0.00030 | 0.00030 |
| Avg error | 0.000283 | 0.00027 | 0.00028 | 0.00027 | 0.00027 | 0.00027 |
| Min error(%) | 0.052918 | 0.00295 | 0.05292 | 0.02373 | 0.02373 | 0.00295 |
| Max error(%) | 12.54795 | 13.65941 | 13.53543 | 14.01645 | 14.01645 | 13.59351 |
| Avg error(%) | 3.3718 | 4.08900 | 3.48000 | 4.05340 | 4.86260 | 4.25540 |

process itself. These complications may manifest as oscillations, divergence, or convergence to incorrect reciprocals. Where efficiency is calculated as:

$$\text{Efficiency} = \frac{\text{Total number of correct converging values}}{\text{Total number of outputs}} \tag{7}$$

The NRD implementation displayed outstanding accuracy, with an average absolute error of just 1.08e-05, even with a maximum of 14 iterations. This high precision was consistently maintained throughout computations. When compared to other methods, Daniel Kho et al. [3] achieved an accuracy of 0.3, while Peter Malik [8] reached 1.1973e-05. In contrast, our design exhibited even higher accuracy at 1.08e-05. This comparison demonstrates the superior performance of our design in terms of accuracy.

*5) Error Analysis:* While evaluating the effectiveness of a method, it is important to analyze all the errors. The current study undertakes a comparative examination of method competence, as delineated in Table II, elucidating the effectiveness of each strategy. The minimum absolute error is reduced from 0.00058 to 0.00027 when the terminating number of iterations is changed from 11 to 12. There is no further change in the minimum error even if the terminating number of iterations is increased beyond 12. The average percentage error is also lower when the number of iterations is limited to 12. Figure 3 offers a detailed analysis of a model's output accuracy across six successive iterations, measured by the number of results that fall within predefined error thresholds of 1%, 2%, and 3% plotted as a bar graph. In each iteration, the green segments of the bars reflect the number of outputs whose deviation from the true value is within the tightest margin of 1%. As the acceptable margin of error widens to 2% and then to 3%, indicated by the orange and red segments, respectively, we see an incremental increase in the count of converging outputs. This modest increase in the number of converging outputs as the error tolerance broadens from 1% to 3% indicates that the bulk of the model's outputs are concentrated near the precise value, signifying a high degree of accuracy. The data also suggest that the model maintains a high level of precision, with output errors clustering within a narrow range, and that outputs become increasingly less likely as the error threshold expands,

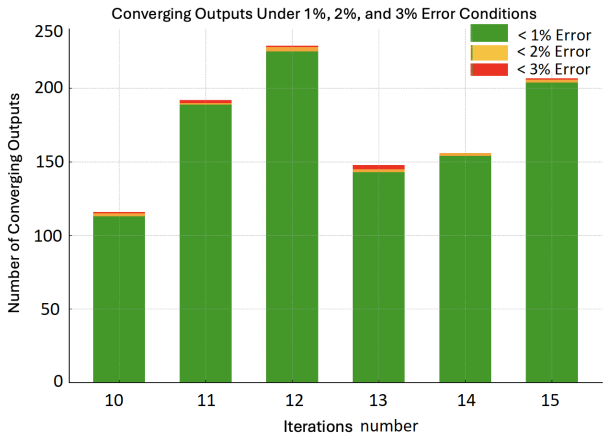



Fig. 3: Converging Outputs Under 1%, 2%, and 3% Error Conditions

highlighting the model's precision-oriented performance.

One more key observation is that when the number of iterations varied from 10 to 14, there is a fluctuation in the minimum and maximum values of the absolute error for the converging values. The minimum and maximum absolute error values for 10 iterations were found to be of the order e-9 and e-6 respectively, which signifies a negligible error. In the subsequent iterations, the maximum and minimum values varied according to the orders e-1 and e-3.

Figure 4 demonstrates the number of correctly predicted values changes as the allowable error margin (from 1% to 30%) increases, across five different iterations (10 to 14). Each line represents a different iteration, showing the growth in the count of predictions of those values that converge with the reference values under increasing error threshold levels. This analysis helps in identifying the suitability of the proposed design for error-tolerant applications such as video processing in entertainment applications, video gaming, etc.

*6) Resource Utilization:* The table III illustrates resource utilization comparison among various methods for implementing a certain process. The methods are characterized by their usage of Look-Up Tables (LUTs), Flip-Flops (FFs), and

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.

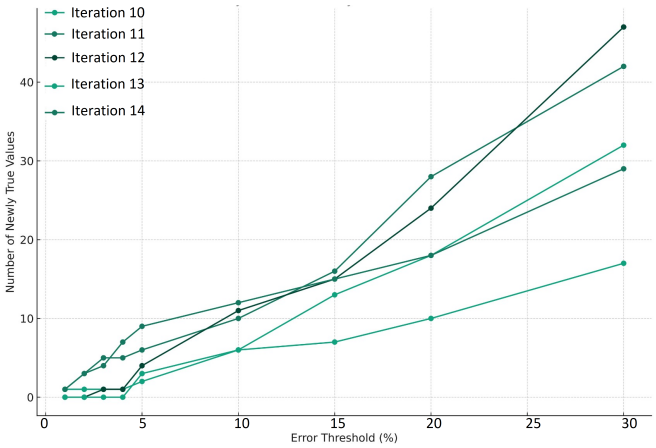



Fig. 4: Covergence versus Error Threshold

Digital Signal Processors (DSPs). Noteworthy observations include the method by Libessart et al. [4] utilizing 9 DSPs, Y. Li and W. Chu's pipelined approach [9] utilizing a significant number of LUTs and FFs, and sS. Zafar's method [10] has the highest LUTs and FFs usage. Notably, the Output-registered FRSR method [3] employs a moderate amount of resources, particularly with 12 DSPs. In contrast, our design demonstrates a more efficient resource usage, with lower LUTs and FFs compared to most methods while maintaining a relatively moderate DSP usage.

*7) Power Consumption:* Our module power consumption is reported to be about 1.82441 mW. This is through the use of fixed-point arithmetic, over the floating-point operations while maintaining computational accuracy. By implementing streamlined conditional logic, iterations cease upon convergence or reaching a preset limit, effectively minimizing redundant cycles.

*8) Robustness:* The challenge comes from the derivative $f'(x_0)$ of the function at the current approximation. If $f'(x_0)$ is near zero, division by zero terminates the iteration process or leads to significant inaccuracies in subsequent iterations.

### *B. Implementation Challenges*

When reciprocals are being calculated using the Newton-Raphson method, achieving accurate convergence largely depends on perfect initial guesses ($X_o$). The wrong $X_o$ values may lead to divergence with the iterative process failing to converge towards the true value of reciprocal ($X_{\text{inv}}$). It is also not easy to work out for inputs which are close to either zero or one due to the minute nature of the derivative and this may impede progress and possibly make the procedure drift away from its direction. Further, another challenge is in synchronizing inverse valid signal with subsequent input processing. Whenever a certain output fails to converge, delays in the inverse valid signal can cause later-inputs acceptance to be slowed down, thereby increasing complexity in implementation.

TABLE III: Resource Utilization

| Method | Length | LUTs | FFs | DSPs |
|---|---|---|---|---|
| In [4] | 32 bits | 203 | - | 9 |
| Y. Li & W. Chu (Pipelined) [9] | 32 bits | 408 | 675 | 0 |
| S. Zafar [10] | 8 bits | 1939 | 748 | 2 |
| Output-registered FRSR [3] | 32 bits | 635 | 82 | 12 |
| Our design | 32 bits | 322 | 84 | 7 |

## VI. Conclusion

The proposed digital sub-system design for reciprocal computation exhibits encouraging characteristics including enhanced precision, reduced resource consumption, and decreased latency. The optimized implementation demonstrated a remarkable 54.55% improvement over conventional methods, resulting in a reduction of latency to 5 clock cycles. The highest achievable throughput rates were 10.10 million operations per second (MOPS). Furthermore, considerable effort was devoted to ensuring that power consumption remained below 1.82441 milliwatts, an approximate 25% reduction compared to previous FPGA implementations. The integration of sophisticated hardware implementations and precision enhancement methodologies with the Newton-Raphson algorithm signifies a critical milestone in the pursuit of attaining maximum computational efficiency and accuracy. This research not only underscores the resilience and expandability of our methodology but also establishes a strong groundwork for subsequent investigations that seek to augment the accuracy and speed of FPGA computations in practical contexts.

## References

[1] M. M. Asad, I. Marouf, Q. A. Al-Haija, and A. AlShuaibi, "Performance analysis of 128-bit modular inverse based extended euclidean using altera fpga kit," *Procedia Computer Science*, vol. 160, pp. 543–548, 2019.

[2] D. Piso and J. D. Bruguera, "Variable latency goldschmidt algorithm based on a new rounding method and a remainder estimate," *IEEE Transactions on Computers*, vol. 60, no. 11, pp. 1535–1546, 2010.

[3] C. Daniel Kho, M. Ahmad Fauzi, and S. Lim, "Hardware implementation of low-latency 32-bit floating-point reciprocal," *J Electr Electron Syst*, vol. 7, no. 278, pp. 2332–0796, 2018.

[4] E. Libessart, M. Arzel, C. Lahuec, and F. Andriulli, "A scaling-less newton–raphson pipelined implementation for a fixed-point reciprocal operator," *IEEE Signal Processing Letters*, vol. 24, no. 6, pp. 789–793, 2017.

[5] D. Piso and J. D. Bruguera, "A new rounding method based on parallel remainder estimation for goldschmidt and newton-raphson algorithms," in *2014 17th Euromicro Conference on Digital System Design*. IEEE, 2014, pp. 639–642.

[6] E. Hertz, N. Thuning, L. Bärring, B. Svensson, and P. Nilsson, "Algorithms for implementing roots, inverse and inverse roots in hardware," 2016.

[7] U. Kucukkabak and A. Akkas, "Design and implementation of reciprocal unit using table look-up and newton-raphson iteration," in *Euromicro Symposium on Digital System Design, 2004. DSD 2004.* IEEE, 2004, pp. 249–253.

[8] P. Malik, "High throughput floating-point dividers implemented in fpga," in *2015 IEEE 18th International Symposium on Design and Diagnostics of Electronic Circuits & Systems*. IEEE, 2015, pp. 291–294.

[9] Y. Li and W. Chu, "Implementation of single precision floating point square root on fpgas," *Proceedings. The 5th Annual IEEE Symposium on Field-Programmable Custom Computing Machines Cat. No.97TB100186)*, pp. 226–232, 1997.

[10] A. Nanhe, G. Gawali, S. Ahire, and K. Sivasankaran, "Implementation of fixed and floating point square root using nonrestoring algorithm on fpga," *International Journal of Computer and Electrical Engineering*, vol. 5, no. 5, p. 533, 2013.

Authorized licensed use limited to: Indian Institute of Technology (Ropar). Downloaded on October 11,2026 at 08:51:02 UTC from IEEE Xplore. Restrictions apply.